

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático II - Simulação de Escalonador de Processos

BCC202 - Estrutura de Dados I

Ana Júlia Barros Araújo

Professor: Pedro Henrique Lopes Silva e Karla Alexsandra de Souza Joriatti

Ouro Preto
5 de fevereiro de 2026

Sumário

1	Introdução	1
1.1	Especificações do problema	1
1.2	Considerações iniciais	1
1.3	Ferramentas utilizadas	1
1.4	Especificações da máquina	1
1.5	Instruções de compilação e execução	1
2	Desenvolvimento	3
2.1	TAD's	3
2.1.1	Processo	3
2.1.2	FilaProcesso	3
2.2	Criação da fila de processos	3
2.2.1	Primeira Entrada	3
2.2.2	Criação da Fila	3
2.2.3	Criação dos Processos	3
2.3	Preenchimento da fila de processos	3
2.4	MergeSort	4
2.5	Escalonamento	4
2.5.1	Adiciona à fila	4
2.6	Impressão do log da CPU	4
3	Experimetos	5
4	Resultados	6
5	Considerações Finais	7

1 Introdução

O trabalho consistiu na implementação de um simulador de escalonador de processos, seguindo um modelo de fila de prioridades com tempo máximo de execução por ciclo (100 ciclos). O sistema foi desenvolvido em C, com separação em módulos (.h e .c), visando modularidade, clareza e ausência de memory leaks.

1.1 Especificações do problema

O código pretende receber uma lista de n Processos (com ID, tempo de chegada, nível de prioridade e quantidade de ciclos de CPU necessários), ordená-los primeiramente por prioridade e, secundariamente, por ordem de chegada. Em seguida, deve simular um escalonador simples. Ao final, deve-se apresentar ao usuário uma lista de atendimento indicando a ordem em que os processos usaram a CPU. Nessa lista, deve constar o ID, a prioridade naquele momento e a quantidade de ciclos que o processo permaneceu na CPU

1.2 Considerações iniciais

Algumas ferramentas foram utilizadas durante a criação deste projeto:

- Ambiente de desenvolvimento do código fonte: Visual Studio Code. ¹
- Linguagem utilizada: C.
- Ambiente de desenvolvimento da documentação: Overleaf L^AT_EX. ²

1.3 Ferramentas utilizadas

Algumas ferramentas foram utilizadas para testar a implementação, como:

- *GCC*: ferramentas de análise estática do código.
- *Valgrind*: ferramentas de análise dinâmica do código.

1.4 Especificações da máquina

A máquina onde o desenvolvimento e os testes foram realizados possui a seguinte configuração:

- Processador: Intel Core i5.
- Memória RAM: 8Gb.
- Sistema Operacional: Ubuntu 24.04 LTS.

1.5 Instruções de compilação e execução

Para a compilação do projeto, basta digitar:

Compilando o projeto

```
gcc -c filaprocessos.c -Wall gcc -c tp.c -Wall gcc filaprocessos.o tp.o -o exe
```

Usou-se para a compilação as seguintes opções:

- *-c*: traduz o arquivo .c em um arquivo objeto (.o), mas não cria o executável final ainda
- *-o*: indica que o termo seguinte é o nome do arquivo executável
- *-Wall*: mostra todos os possíveis warnings do código.

¹Visual Studio Code está disponível em <https://code.visualstudio.com>

²Disponível em <https://www.overleaf.com/>

Para a execução do programa basta digitar:

```
cd caminho_até_o_arquivo_de_entrada ./exe
```

2 Desenvolvimento

O programa foi implementado tentando-se fazer o uso do máximo de funções necessárias para que a `main()` ficasse o mais limpa possível e também para que o arquivo das funções ficasse mais legível e simples de navegar.

2.1 TAD's

Foram desenvolvidos dois Tipos Abstratos de Dados para essa implementação, sendo eles o `Processo` e o `FilaProcesso`.

2.1.1 Processo

Esse TAD tem a função de guardar todas as informações de um processo, sendo elas:

- ID: número de identificação do processo;
- T: tempo de chegada do processo. Essa variável é a única float do TAD;
- P: número que indica o nível de prioridade do processo;
- C: número de ciclos que o processo precisa ficar na CPU.

2.1.2 FilaProcesso

O TAD em questão possui um vetor de `Processos` e uma variável inteiro responsável por armazenar o tamanho do vetor de `Processos`

2.2 Criação da fila de processos

Essa seção tratará de um conjunto de três funções que trabalham sequencialmente para criar um vetor com a quantidade necessária de processos que serão futuramente informados pelo usuário.

2.2.1 Primeira Entrada

A função `primeiraEntrada()` lê e retorna para uma variável localizada na `main()` o valor inteiro referente à quantidade de processos que o usuário informará ao programa.

2.2.2 Criação da Fila

O algoritmo de `*criarFila()` utiliza recursos da biblioteca `jstlib.h` para criar uma variável de tipo `FilaProcesso`, além de armazenar na mesma o tamanho que ela deve ter e retorná-la à uma variável ponteiro no `main()`.

2.2.3 Criação dos Processos

Dentro da função `*criarFila()` é chamada a função `*criarProcesso()`, que é responsável por criar um vetor com a quantidade necessária de `Processos` e retorná-lo para o ponteiro que faz parte do tipo `FilaProcesso`.

2.3 Preenchimento da fila de processos

A função `segundaEntrada()` tem como objetivo preencher a variável `fila` com as informações fornecidas pelo usuário e para isso é usado um `for()` que se repete a quantidade de vezes necessárias para receber e armazenar todas as informações necessárias.

2.4 MergeSort

Essa parte do código é chamada dentro da função `segundaEntrada()` e usa do método MergeSort para ordenar o vetor de processos. Dentro desta função é também utilizada a função `comparar()` que ajuda a função principal a ordenar os processos com base em seus níveis de prioridade e, em caso de empate, de acordo seu tempo de chegada.

2.5 Escalonamento

A função `escalonador()` gerencia a execução dos processos, limitando seu tempo na CPU a 100 ciclos por vez. Após cada entrada na CPU, informações do processo em questão são armazenados em uma lista de log na CPU. Se um processo não termina, sua prioridade é incrementada, seus ciclos restantes são atualizados e ele é reinserido na fila usando a função `adicionaFila()`.

2.5.1 Adiciona à fila

Essa função move o último processo executado para o final da fila e confere se ainda restam ciclos a serem executados no mesmo. Se ainda restarem ciclos, então é feito um `insertionSort` e o processo é movido para a posição correta com base em seu nível de prioridade. Se não restarem mais ciclos, então a fila perde sua última posição, onde o processo em questão está localizado.

2.6 Impressão do log da CPU

Esta função é responsável por imprimir ao usuário a lista de todos os processos que entraram na CPU na ordem em que foram executados apresentando também sua prioridade no momento e também por quantos ciclos permaneceu na CPU.

3 Experimetos

O trabalho atingiu seus objetivos, implementando um simulador de escalonador de processos com filas e métodos de ordenação. As principais dificuldades foram:

- Integrar a ordenação inicial (MergeSort) com a ordenação de reinserção (InsertionSort) sem perda de desempenho;
- Garantir a correta atualização dos ciclos restantes e prioridades após cada execução parcial;
- Garantir a correta atualização dos ciclos restantes e prioridades após cada execução parcial;
- Assegurar a liberação completa de memória em todas as situações, incluindo realocações.

O uso de modularização (arquivos separados .h e .c) facilitou a organização e a manutenção do código. A experiência reforçou a importância de planejar estruturas de dados e algoritmos antes da codificação, especialmente em sistemas que simulam comportamento dinâmico.

4 Resultados

O programa foi testado com diferentes entradas, incluindo o exemplo fornecido no enunciado, e produziu saídas consistentes com o comportamento esperado do escalonador:

- Processos de maior prioridade são executados primeiro;
- Processos com mesma prioridade são ordenados pelo tempo de chegada inicialmente;
- Ciclos limitados a 100, com reinserção e incremento de prioridade;
- Ausência de memory leaks confirmada com valgrind.

5 Considerações Finais

O desenvolvimento deste trabalho foi desafiador e instrutivo. A parte mais interessante foi a modelagem do escalonador, simulando o comportamento de um kernel real, com prioridades e ciclos de CPU. A implementação do MergeSort modificado para ordenação múltipla (prioridade e tempo) exigiu atenção aos detalhes de comparação. Um ponto desafiador foi garantir que a reinserção dos processos na fila mantivesse a ordem correta de prioridade sem afetar a lógica de execução. A ferramenta valgrind foi essencial para assegurar a gestão correta da memória. Adquiri maior familiaridade com:

- Alocação dinâmica e realocação de vetores em C;
- Implementação de algoritmos de ordenação com critérios compostos;
- Simulação de sistemas com múltiplos estados e transições;